Applied Soft Computing 132 (2023) 109860

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

LayerLog: Log sequence anomaly detection based on hierarchical semantics

Chunkai Zhang [a,*], Xinyu Wang [a], Hongye Zhang [a], Jiahua Zhang [a], Hanyu Zhang [b], Chuanyi Liu [a], Peiyi Han [a,*]

[a] Harbin Institute of Technology(Shenzhen), Shenzhen, 518000, China
[b] New York Stock Exchange LLC, Wall Street, NY, 10005, United States

ARTICLE INFO

Article history:
Received 30 November 2021
Received in revised form 24 July 2022
Accepted 18 November 2022
Available online 29 November 2022

Keywords:
Anomaly detection
Log parser
"Word-Log-Log Sequence" hierarchy

ABSTRACT

System logs record the running status of systems, and log anomaly detection can help locate anomalies timely to reduce error time and ensure normal operation. Logs in text format contain abundant semantic information. Still, the methods based on log count vector and log key sequence do not take into account the semantics of log data, causing a high missing detection rate. And existing log semantics-based methods fail to fully consider the semantic information among words, logs, and log sequences. Besides, current methods require the usage of Log Parser in preprocessing, which could negatively impact detection accuracy and cause semantic loss. In this paper, we discover the three-layered structure of log data, named the "Word Log-Log Sequence" hierarchy, and propose LayerLog, a novel framework for log sequence anomaly detection based on the hierarchical semantics of log data. Without Log Parser in preprocessing phase, LayerLog can effectively extract semantic features from each layer and is the first framework to consider the semantics of words, logs, and log sequence. What is more, LayerLog can detect execution order anomaly, operation anomaly, and incomplete anomaly of log sequence simultaneously in an end-to-end way. We have evaluated sufficient experiments on two commonly-used public datasets and the experimental results confirm the effectiveness of LayerLog.
© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Computer systems usually generate a large number of system logs, which record the operation of systems in textual form, and indicate the status of different critical points and important activities [1]. Therefore, log anomaly detection can help locate anomalies and perform cause analysis to reduce error time and ensure the normal running of systems [2].

Logs are arranged in order of generation time to form a log sequence. But, not all logs in an abnormal sequence are anomalies. Especially, a log sequence consisting of individual normal logs may also be abnormal. Therefore, the log data is contextual and we need to detect anomalies from the log sequence perspective [3]. Fig. 1 is an example of a normal log sequence with five logs represented as {L1, L2, L3, L4, L5}. We have summarized three types of log sequence anomalies, as shown below:

1. Execution Order Anomaly: All logs in the log sequence are normal, but the execution order deviates from the normal pattern. (e.g., {L1, L4, L2, L3, L5} or {L1, L3, L4, L5, L2} with incorrect order)
2. Operation Anomaly: Log sequence contains one or more logs that should not have occurred in the log sequence. These logs may be normal or abnormal, but no matter what, they should not appear in this log sequence. (e.g., {L1, L2, L3, L#, L4, L5} with L# occurring wrongly)
3. Incomplete Anomaly: The linear logical correlation among logs is incomplete. (e.g., {L1, L2, L3, L5} without L4 occurring)

Currently, the methods of log sequence anomaly detection can be roughly divided into three categories: Log event count-based methods (e.g., LR [4], SVM [5], PCA [6], IM [7], LogClustering [8], LSTM-AE [9]), Log key sequence-based methods (e.g., Deeplog [10], logkey2vec [11]) and Log templates-based methods (e.g., LogAnomaly [3], LogRobust [12]). We have summarized the drawbacks of the existing methods as follows:

1. Almost all of the above methods rely on log parsers which convert unstructured log data into structured log templates or log keys. However, due to the various log data formats in different systems, a fixed Log Parser is unsuitable for all logs [13]. Log parsers may cause the loss of semantic information and their robustness and accuracy greatly affect the performance of log anomaly detection.

* Corresponding authors.
E-mail addresses: ckzhang@hit.edu.cn (C. Zhang), hanpeiyi@hit.edu.cn (P. Han).

https://doi.org/10.1016/j.asoc.2022.109860
1568-4946/© 2022 Elsevier B.V. All rights reserved.

```
L1:  2015-08-10 17:52:39 - Accepted socket connection from /10.10.34.11:33644
L2:  2015-08-10 17:52:39 - Connection request from old client /10.10.34.11:33644;
     will be dropped if server is in r-o mode
L3:  2015-08-10 17:52:39 - Client attempting to establish new session at
     /10.10.34.11:33644
L4:  2015-08-10 17:52:39 - Established session 0x34f0557885e0003 with negotiated
     timeout 10000 for client /10.10.34.11:33644
L5:  2015-08-10 17:52:45 - Closed socket connection for client /10.10.34.11:33644
     which had sessionid 0x34f0557885e0003
```

Fig. 1. A normal log sequence composed of five logs.

2. Both Log event count-based methods and Log key sequence-based methods ignore the semantic information in logs. Log templates-based methods simply adopt the word embedding model to map words into word vectors. A log is composed of words, so the semantics and context of words determine the semantics of the log. The existing methods ignore the impact of words' order and importance degree on log semantics. Besides, we consider it important to extract semantic information of the log sequence sufficiently at different layers rather than just the semantic information among words.

A log sequence is composed of logs, and a log is composed of words, thus forming the three layers of the hierarchical structure called the "Word-Log-Log Sequence" hierarchy. From the above analysis, the semantics of each layer will affect the anomaly detection result. This paper proposes a novel framework for log sequence anomaly detection based on hierarchical semantics named LayerLog. LayerLog can extract the dynamic semantic information from each layer and map them to the fixed dimension semantic vectors for representation. We have evaluated LayerLog on the HDFS [14] and BGL [15] datasets. The experimental results confirmed that LayerLog has performed effectively for log sequence anomaly detection.

The main contributions of this paper are as follows:

(1) We point out the drawbacks of log parser-based anomaly detection methods and consider extracting semantic information directly from the raw logs instead of converting them to log templates.
(2) We find the "Word-Log-Log Sequence" hierarchy of log data and propose LayerLog, a novel framework based on hierarchical semantics for log sequence anomaly detection. LayerLog does not rely on any Log Parses during the preprocessing phase and can effectively extract semantic features from each layer.
(3) We have evaluated LayerLog on two public datasets. The experimental results confirm the effectiveness of LayerLog for anomaly detection.

The rest of this paper is organized as follows: Section 2 reviews the related work. Section 3 illustrates the design of LayerLog. Section 4 shows the experimental results that our method performs effectively on several benchmarks. Finally, the conclusion is drawn in Section 5.

2. Related work

2.1. Log parsing

The log parser is used as a preprocessing work, and its accuracy greatly impacts the log anomaly detection results. The primitive method generates regular expressions based on source codes to parse logs [16]. Currently commonly used log parsing methods can be divided into several categories. 1) Clustering-based methods: The criterion for clustering is usually similarity or frequency. LKE [17] converts the free-form text of the log into log keys and then clusters the log keys. LogSig [18], LogMine [19] groups similar logs into the same cluster to generate the same log template. iPLoM [20] mainly calculates the number of occurrences of each word at each log position. Next, frequent words are selected and synthesized into event candidates. Finally, select some candidates as log templates. 2) Heuristic-based methods: Drain [21] utilizes a fixed-depth prefix tree structure to represent log messages and effectively extract log templates. FT-tree [22] identifies the longest combination of frequently occurring words from the system log message to generate log templates. Spell [23] is an online real-time dynamic log analysis method based on LCS (Longest Common Subsequence). It realizes the dynamic acceptance of log input, real-time processing of the input, and continuous generation of new log templates. 3) Dictionary-based: The general idea is to construct a dictionary from existing templates and apply it to generate and update new templates. For example, Logparse [24] builds a word classifier to distinguish log messages by identifying the variable and constant parts of a log. Logram [25] leverages n-gram dictionaries instead of single tokens to split the constant n-gram and variable n-gram.

Among almost log anomaly detection methods, there exists Log Parser that converts unstructured log data into structured log templates or log keys [26]. However, due to the various log data formats in different systems, a single Log Parser is unsuitable for all log types [13]. The empirical study shows that there are around 20%–45% logs changed throughout the software system lifetime [12]. Obviously, the log parse methods based on the log template have no way to address these changing logs. LayerLog does not rely on any log parser and has better robustness with its "Word-Log-Log Sequence" hierarchy.

2.2. Log anomaly detection

2.2.1. Log event count-based methods

Large online computer systems may generate various logs, making it very difficult to identify them manually. Initially, scholars proposed some rule-based methods to identify anomalous events. However, this requires significant prior knowledge and only applies to specific scenarios. Subsequently, several studies have used machine learning methods for anomaly detection (Decision Tree (DT) [27,28], Support Vector Machines (SVM) [5], and Logistic Regression (LR) [4]). Since log sequences generated by system programs have specific patterns, PCA is used to find log patterns in the dimensions of the event count vector to generate the normal and abnormal spaces [6,29]. Some studies also define the concept of invariant mining (IM), stating that if a

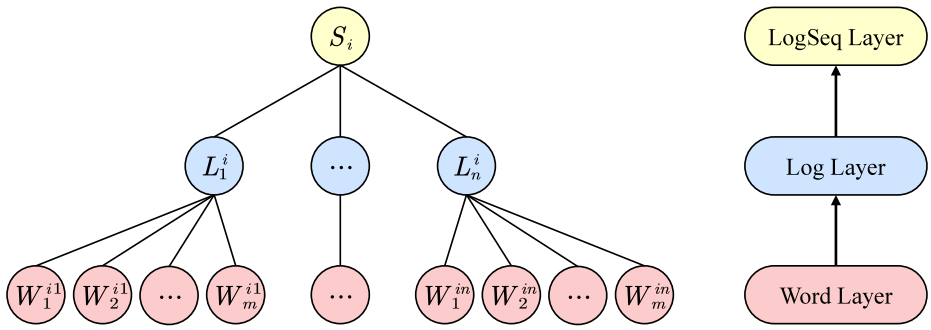

Fig. 2. "Word-Log-Log Sequence" hierarchy of log data.

new instance deviates from an invariant, then it is considered an anomaly [7]. These models firstly utilize Log Parses to transform raw logs into log events and then represent them by Log Event Count [30]. Each dimension value in the Event Count Vector represents the number of corresponding events. However, the above methods cannot reflect the execution order of logs in a log sequence.

2.2.2. Log key sequence-based methods

Log key sequence-based methods have considered the execution time of logs because it is formed by log keys in the order of execution time. Deeplog [10] utilizes word2vec [31] to generate word vectors and uses LSTM to predict the next log key. If the real log key is not in the log key set consisting of the predicted Top-N log keys, this log sequence will be judged as an anomaly. Logkey2vec [11] maps log key sequences to the fixed dimension vectors and then uses CNN binary classification model to identify abnormal log sequences. However, these methods require the length of the log key sequence to be specified in advance. If the length is too short, the log key sequence will be truncated, which artificially causes an incomplete anomaly.

2.2.3. Log template-based methods

Neither Log Event Count-based methods or Log key sequence-based methods takes full account of the contents of the raw logs. Therefore, several methods extract semantic information from the logs for anomaly detection. Log2Vec [32] uses a word2vec-based model to convert raw log messages into semantic vectors. Direct vectorization of raw log messages without processing can result in a redundant and incomplete understanding of semantic information. LogAnomaly [3] utilizes FT-Tree [22] to parse logs, and proposes the template2vec structure to extract semantic information of log templates. LogRobust [12] designs a combination of Fastext [33] and TF-IDF to map words into word vectors and then adds them as the embedding representations of log templates. Both LogAnomaly and LogRobust utilize Bi-LSTM with an attention model to detect anomalies. However, the above methods can only generate an embedding representation for each word and ignore contextual information. LogBERT [34], SwissLog [35] and NeuralLog [36] utilize the BERT model [37] to encode log templates to obtain log representations. But the existing log semantics-based methods only extract semantic information from the perspective of the logs consisting of words. LayerLog proposes the three layers of hierarchical structure and considers comprehensive and complex semantic information.

3. Design of LayerLog

3.1. Overview

This paper proposes a novel framework for log anomaly detection based on hierarchical semantics named as LayerLog. The layer of words, logs, and log sequences are called "Word Layer", "Log Layer" and "LogSeq Layer" respectively. We assume that the *ith* log sequence consists of *n* logs, and the *jth* log consists of *m* words. Let S_i be the *ith* log sequence, L_j^i be the *jth* log in S_i, and W_k^{ij} be the *kth* word in L_j^i. So we can get $S_i = \left\{L_1^i, L_2^i, \ldots, L_n^i\right\}$ and $L_j^i = \left\{W_1^{ij}, W_2^{ij}, \ldots, W_m^{ij}\right\}$. The representation of the "Word-Log-Log Sequence" hierarchical structure of log data is shown in Fig. 2.

The goal of LayerLog is to extract each layer's semantic information and represent the semantic characteristics of three layers with fixed dimensional semantic vectors, then determine whether the semantic vector of the log sequence is abnormal. The semantic vectors corresponding to Word Layer, Log Layer, and LogSeq Layer are called WordVec, LogVec, and LogSeqVec respectively. In more detail, we represent WordVec of W_k^{ij} as wv_k^{ij}, LogVec of L_j^i as lv_j^i, and LogSeqVec of S_i as sv_i.

The design of our proposed LayerLog is shown in Fig. 3. In Word Layer, LayerLog combines word embedding and important degree calculation (POS [38] and TF-IDF [39]) to generate a corresponding semantic vector WordVec for each word. Then the word representation vectors in the log are passed into Log Layer to form a WordVec sequence. The corresponding semantic vector LogVec is generated through the Bi-LSTM model based on the attention mechanism. In the same way, the log representation vectors are transferred to the LogSeq layer to form a LogVec sequence. The Bi-LSTM model based on the attention mechanism is used again to generate the corresponding LogSeqVec. Finally, LayerLog judges whether the log sequence is abnormal by an anomaly detection module.

3.2. Word layer

The goal of Word Layer is to represent each word in each individual log with a fixed dimension WordVec. The specific implementation of Word Layer is shown in Fig. 4 and the steps to generate wv_k^{ij} are as follows:

Step 1. Raw Log Data Preprocessing

This step divides into two parts: Word Segmentation and Nonsense Word Removal. After segmenting the original log entries, we remove the meaningless symbols (such as punctuation marks and separators) because they have no positive effect on the semantics of words. Note that although in natural language understanding, prepositions (e.g., "from", "to") or quantifiers (e.g., "a", "the") tend to be meaningless, we still retain them. We consider that all words have semantic information during the log analysis process, but the importance degree is different. In addition, we construct corresponding WordVec to retain the whole semantics for compound words such as "PackeResponder", "addStoredBlock", etc. LayerLog does not deliberately segment

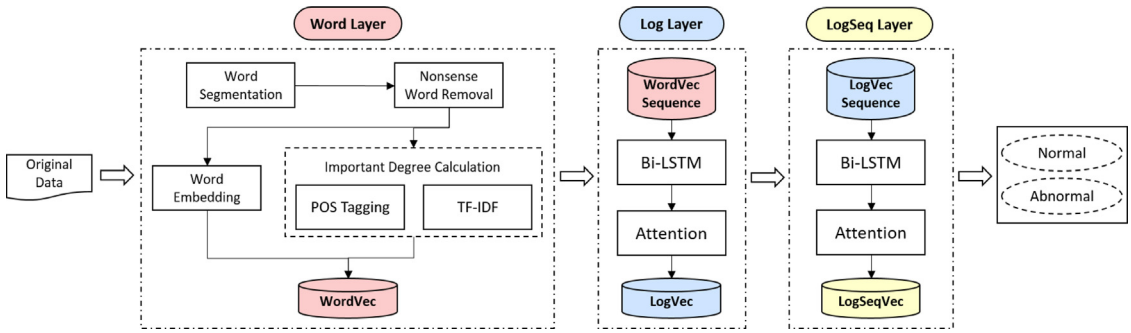

Fig. 3. Framework of LayerLog.

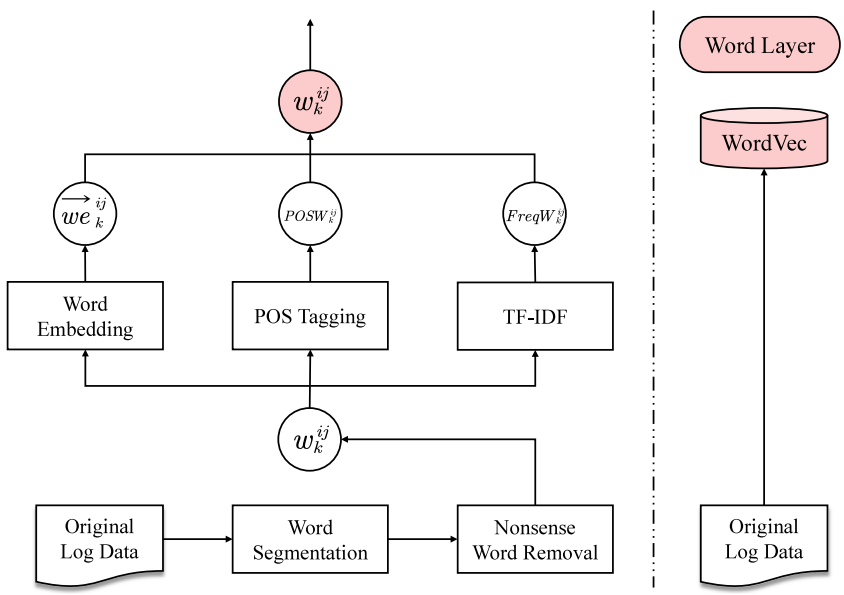

Fig. 4. The specific implementation process of Word Layer.

these compound words and treat them as a specific word since splitting them may cause semantics loss of the whole word.

Step 2. Word Embedding

The word embedding representation maps words into the word vectors. Word2vec [40] is a Google open-source tool for word vector computing and it is a process of mapping sparse word vectors in one-hot form into a dense vector in n dimension by using a one-layer neural network (CBOW or Skip-Gram). LayerLog records the vector of W_k^{ij} through Word2vec as $\overrightarrow{we}_k^{ij}$.

Step 3. Important Degree Calculation

The importance degree of each word in a log is different, as indicated in the following two aspects: (1) Part of Speech (POS) for words is different. In the aspect of natural language understanding, we emphasize that content words are dominant and function words are auxiliary, i.e., the importance degree of content words is usually more significant than the function words. (2) The same word will be more important in some log sequences but less important in others, depending on the context of its surrounding words and its belonging logs. Therefore, we combine POS Tagging with TF-IDF to calculate the important degree of words.

- **Method 1. POS Tagging**
  We utilize NLTK (Natural Language Toolkit) [41] natural language processing library designed by the University of Pennsylvania to mark the POS of words. The corresponding weights of POS are shown in Table 1. We record the weight of w_k^{ij} after POS tagging as $PosW_k^{ij}$. The higher $PosW_k^{ij}$ is, the more important W_k^{ij} is.

Table 1
The corresponding weights of POS. (Abbreviations for POS are following the NLTK Standard).

	POS	Abbr.	PosW
Content Word	Noun	NN	1.5
	erb	VB	1.5
	Adjective	JJ	1.5
	Adverb	RB	1.5
Function Word	Conjunction	CC	0.5
	Determiner	DT	0.5
	Preposition	IN	0.5

- **Method 2. TF-IDF**
  In addition to POS Tagging, word frequency also affects the importance degree of words in logs. LayerLog utilizes the TF-IDF (term frequency-inverse document frequency) [39] algorithm to calculate each word's frequency weight. For the "Word-Log-Log Sequence" hierarchy, this method is based on the following assumption: if a word frequently appears in one log sequence and rarely in other log sequences, then this word is more discriminating and so it has a higher

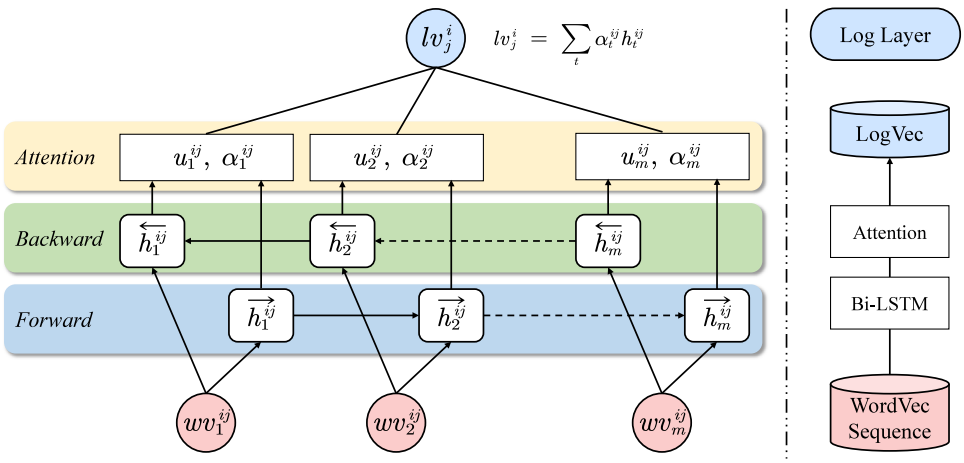

Fig. 5. The specific implementation process of Log Layer.

importance degree. We can firstly obtain *TF* and *IDF* values as follow:

$$TF_W_k^{ij} = \frac{|L^i(W_k^{ij})|}{|S_i|} \tag{1}$$

where $|S_i|$ indicates the number of logs in the *ith* log sequence, and $|L^i(W_k^{ij})|$ indicates the total number of logs in S_i containing W_k^{ij}.

$$IDF_W_k^{ij} = lg\left(\frac{|S|}{|S(W_k^{ij})|}\right) \tag{2}$$

where $|S|$ indicates the total number of log sequences in the dataset. $|S(W_k^{ij})|$ indicates to the total number of S containing W_k^{ij}. Finally, we calculate the *TF-IDF* value $FreqW_k^{ij}$ of W_k^{ij} by the following formula:

$$FreqW_k^{ij} = TF_W_k^{ij} * IDF_W_k^{ij} \tag{3}$$

Step 4. WordVec Generating

We combine word embedding with importance degree to generate WordVec wv_k^{ij} corresponding to W_k^{ij}. The calculation strategy is as follows:

$$wv_k^{ij} = \left(\alpha\, PosW_k^{ij} + \beta\, FreqW_k^{ij}\right) * \overrightarrow{we}_k^{ij} \tag{4}$$

Where α and β are the weight factors to adjust $PosW_k^{ij}$ and $FreqW_k^{ij}$.

3.3. Log layer

Log Layer aims to generate a semantic vector representation *LogVec*. We cannot simply add WordVecs to get LogVec because it ignores the order and syntactic relationship among words. In the Log Layer, we utilize the Bi-LSTM model with an attention mechanism to extract features among WordVec sequences. The Attention-based Bi-LSTM can capture the contextual information among words and automatically adjust the importance degree of words. The specific implementation of the Log Layer is shown in Fig. 5.

3.3.1. Bi-LSTM model

LSTM [42] is a variant of RNN, which overcomes the long-term dependence of RNN by introducing a gating unit to alleviate gradient disappearance. Bi-LSTM combines forward LSTM with backward LSTM to encode bidirectional sequence information and is more suitable for modeling sequential data. The log consists of words and each WordVec corresponding to a word in a log constitutes a WordVec sequence. Bi-LSTM can better capture the bidirectional semantic information of the WordVec sequence.

LayerLog regard the WordVec sequence $\left\{wv_1^{ij}, wv_2^{ij}, \ldots, wv_m^{ij}\right\}$ as the input into the Bi-LSTM model. We record the value of the hidden layer state at time t in forward LSTM as $\overrightarrow{h}_t^{ij}$ and in backward LSTM as $\overleftarrow{h}_t^{ij}$. The calculation of $\overrightarrow{h}_t^{ij}$ and $\overleftarrow{h}_t^{ij}$ can be simplified as $\overrightarrow{h}_t^{ij} = \overrightarrow{LSTM}(wv_t^{ij})$ and $\overleftarrow{h}_t^{ij} = \overleftarrow{LSTM}(wv_t^{ij})$. Finally, we concatenate them as the representation by

$$h_t^{ij} = \left[\overrightarrow{h}_t^{ij},\ \overleftarrow{h}_t^{ij}\right] \tag{5}$$

3.3.2. Attention mechanism

Not all words have the same effect on the semantic representation of logs. We introduce an attention mechanism to capture more complex contextual semantic information to express the log semantics accurately. Firstly, h_t^{ij} is provided with word representation through single-layer *MLP* to obtain the hidden representation u_t^{ij}.

$$u_t^{ij} = tanh\left(W_{ij} h_t^{ij} + b_{ij}\right) \tag{6}$$

Then, LayerLog measures the importance of words $u_t{}^{ij}$ by calculating the similarity to the context vector u_{ij} in the word layer, which is normalized by SoftMax to obtain α_t^{ij}.

$$\alpha_t^{ij} = \frac{exp\left((u_t^{ij})^T u_{ij}\right)}{\sum_t exp\left((u_t^{ij})^T u_{ij}\right)} \tag{7}$$

Finally, the semantic vector of the log can be calculated by:

$$lv_j^i = \sum_t \alpha_t^{ij} h_t^{ij} \tag{8}$$

Based on the Bi-LSTM model with an attention mechanism in the Log Layer, we can obtain the semantic representation LogVec of the log composed of words.

3.4. LogSeq layer

The LogSeq layer targets to generate a semantic vector representation of the log sequence, that is LogSeqVec. A log sequence consists of a series of log events generated in chronological order which are correlated. Therefore, we again utilize the Bi-LSTM model based on an attention mechanism to extract semantic

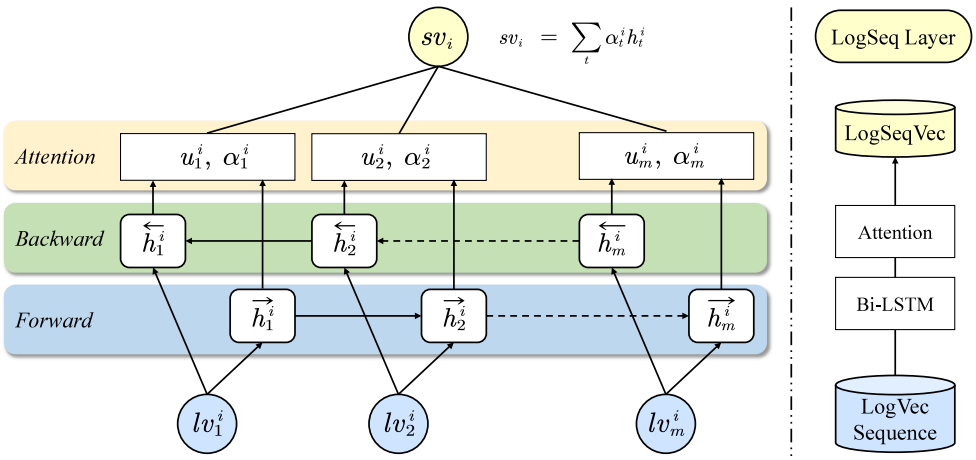

Fig. 6. The specific implementation process of LogSeq Layer.

features among LogVec sequences. The specific implementation process of the LogSeq Layer is shown in Fig. 6.

3.4.1. Bi-LSTM model

In the same way, LayerLog utilizes Bi-LSTM to extract semantic information among logs in a log sequence. At the current time t, h_{t-1}^i represents the hidden layer state value at the previous time $t-1$ and C_t^i represents the cell state value at the current time. We record the value of hidden layer state at time t in forward LSTM as $\overrightarrow{h_t^i}$ and that at time t in backward LSTM as $\overleftarrow{h_t^i}$. Therefore, the calculation method of $\overrightarrow{h_t^i}$ and $\overleftarrow{h_t^i}$ can be simplified as follows: $\overrightarrow{h_t^i} = \overrightarrow{LSTM}\left(lv_t^i\right)$ and $\overleftarrow{h_t^i} = \overleftarrow{LSTM}\left(lv_t^i\right)$. Then, we connect these two vectors to obtain the final representation:

$$h_t^i = \left[\overrightarrow{h_t^i},\ \overleftarrow{h_t^i}\right] \tag{9}$$

3.4.2. Attention mechanism

For different log sequences, the function of the log entries is different. In the same way, we introduce an attention mechanism to capture more contextual semantic information between logs and summarize the expression of these words to form sv_i. Firstly, h_t^i is provided with log representation through single-layer *MLP* to obtain the hidden representation:

$$u_t^i = tanh\left(W_i h_t^i + b_i\right) \tag{10}$$

Then, the importance of log as u_t^i is measured by calculating the similarity with the context vector u_i in Log Layer, and α_t^i is obtained by SoftMax normalization:

$$\alpha_t^i = \frac{exp\left((u_t^i)^T u_i\right)}{\sum_t exp\left((u_t^i)^T u_i\right)} \tag{11}$$

Finally, we calculate the weight of the log representation to obtain sv_i:

$$sv_i = \sum_t \alpha_t^i h_t^i \tag{12}$$

Through the above steps, we can get LogSeqVec for each log sequence as the output of LogSeq Layer, which is the semantic representation of the log sequence composed of logs.

3.5. Anomaly detection module of LayerLog

In LogSeq Layer, we extract the semantic vector *LogSeqVec* for the log sequence, which can help us to judge anomalies. As an anomaly detection framework, LayerLog gets the semantic vector obtained through the "Word-Log-Log Sequence" hierarchy, which can be input into different types of anomaly detection methods for anomaly judgment.

3.5.1. Supervised anomaly detection method

In the supervised anomaly detection methods, we regard the anomaly detection of log sequence as a binary problem (Normal or Abnormal). We utilize the SoftMax function to calculate the probability distribution of a log sequence, and then the detection result corresponding to the maximum conditional probability is selected as the prediction output category. The calculation process is as follows:

$$p(\hat{y}_i|S_i) = SoftMax\left(WS_i + b\right) \tag{13}$$

$$y_i = \arg\max_{\hat{y}_i}\ p(\hat{y}_i|S_i) \tag{14}$$

Where $p(\hat{y}_i|S_i)$ indicates the probability distribution function of the log sequence S_i and y_i indicates the detection result of S_i

In the training phase, we calculate the cross-entropy of prediction outputs and real labels provided by the data set as the loss function:

$$Loss = -\frac{1}{m}\sum_{i=1}^{m} t_i \log p(\hat{y}_i|S_i) \tag{15}$$

Where t_i indicates the one-hot vector of the real label, and m represents the total number of log sequences.

3.5.2. Semi-supervised anomaly detection method

In practical scenarios, supervised methods are often not applicable due to the need for a large amount of prior knowledge and sample labels. Therefore, we always adopt semi-supervised methods, only using positive sample data to train the model. One-Class SVM [43] is a commonly used semi-supervised anomaly detection method. OC-SVM has a good effect on the novelty detection of high-dimensional data. In the training phase, we use normal data to train the model and minimize the volume of the hypersphere to get the boundary. The constraint formula is as follows:

$$\begin{aligned} &\min_{o,r} V(r) + C\sum_{i=1}^{m}\zeta_i \\ &s.t.\quad \begin{cases} \|S_i - o\|^2 \le r^2 + \zeta,\ i = 1,\ldots,n \\ \zeta \ge 0,\ i = 1,\ldots,n \end{cases} \end{aligned} \tag{16}$$

where parameter o is the center of the hypersphere, and r is the radius of the hypersphere. Then, minimize the volume $V(r)$ of the hypersphere to obtain the distribution space of the normal data. In the testing phase, the anomaly score is defined as follows:

$$\text{Score}(X_i) = \|X_i - o\|^2 \tag{17}$$

By calculating the distance between the sample X_i and the center point o, we can judge whether X_i is within the class. Furthermore, we can determine whether X_i is abnormal.

4. Evaluation

4.1. Experimental design

4.1.1. Datasets

Experiments are carried out on HDFS and BGL datasets. The details of these two datasets are as follows:

- HDFS dataset: HDFS dataset [14] contains 11,175,629 logs generated by Hadoop which is collected from more than 200 Amazon EC2 nodes. Sessions of the HDFS dataset are grouped by blockID, that is, each session corresponds to a blockID. Among 11,175,629 logs, there are 575,061 sessions in total. The normal or abnormal tags of these sessions have been marked by Hadoop domain experts. The number of normal sessions is 558,223, and the number of abnormal sessions is 16,838.
- BGL dataset: BGL dataset [15] is generated by a Blue Gene/L supercomputer, which consists of a 128 K processor and is deployed at Lawrence Livermore National Laboratory (LLNL). BGL dataset contains 4,747,963 logs, of which 348,460 are abnormal. Unlike HDFS, logs in BGL do not contain a specific sessionID. So we divide the BGL dataset by session size according to the execution time order of logs.

4.1.2. Evaluation metrics

To measure the effectiveness of LayerLog in log sequence anomaly detection, we use precision, recall, and F1-score to evaluate.

- Precision (P): the percentage of true log sequence anomalies detected among all anomalies detected. $P = \frac{TP}{TP+FP}$.
- Recall (R): the percentage of true log sequence anomalies detected among all anomalies actually. $R = \frac{TP}{TP+FN}$.
- F1-Score (F_1): the harmonic mean of Precision and Recall. $F_1 = \frac{2*P*R}{P+R}$.

TP is the number of abnormal log sequences correctly detected. FP is the number of normal log sequences wrongly detected as anomalies. FN is the number of abnormal log sequences wrongly detected as normal.

4.1.3. Environment

We implement LayerLog with Python 3.6 and TensorFlow 1.13 on a Linux server with Intel(R) Core(TM) i9-9820X CPU @ 3.30 GHz and 96G memory.

4.2. Evaluation of overall performance

In this section, the performance of LayerLog is verified on the HDFS and BGL datasets. We compare LayerLog with 55, SVM, PCA, IM, LogClustering, LSTM-AE (Log event count-based method), DeepLog (Log key sequence-based method), and LogAnomaly (log template-based method). Table 2 shows the comparison results on HDFS and BGL datasets.

It can be seen from the experimental results that LayerLog performs better than other methods. It achieves F1-score of 0.988 on the HDFS dataset and 0.985 on the BGL dataset. Let us simply analyze the experimental results. Log event count-based methods cannot achieve high precision and high recall simultaneously. For example, the Precision of LR and SVM on HDFS and BGL datasets are even up to 1, while their Recall are low by contrast, resulting in a low F1-score. Meanwhile, the robustness of log event count-based methods is poor, for the two datasets' evaluation criteria are quite different. For instance, the F1-score of SVM on the HDFS dataset can reach 0.926, but on the BGL data set is only 0.71. Log key sequence-based methods generally perform better than Log event count-based methods. The F1-score of DeepLog on two datasets is more than 0.9, which emphasizes the consideration of the execution order of logs for log sequence anomaly detection and verifies the better robustness performance. Log template-based methods achieve almost perfect experimental results, which illustrates that understanding the execution content of log sequences from the semantic perspective improves the ability to detect three different kinds of anomalies of log sequences. The performance of LayerLog is better than that of LogAnomaly, and its F1-score is nearly 3 percentage points higher than that of LogAnomaly. It confirms that LayerLog can extract more abundant semantic features from words, logs, and log sequences instead of log templates so that LayerLog can learn more accurate hierarchical semantic expressions and achieve the best accuracy.

4.3. Evaluation of online testing

The systems will generate new log formats during running. As the trained model learns a fixed log sequence semantic pattern from the training data, the accuracy may be reduced when detecting new log sequences. Therefore, we conduct an online test on the BGL dataset and compare LayerLog with DeepLog and LogAnomaly to test the adaptability of the anomaly detection model to new log data. The specific experimental result is shown in Table 3.

It can be seen from the experimental results that LayerLog has a strong ability to adapt to new log formats. Since both DeepLog and LogAnomaly require a log parser to convert the logs into log templates, which leads to a significant decrease in the performance of log anomaly detection. By contrast, LayerLog does not rely on any log parsers, which avoids the negative impact of Log Parser on log anomaly detection. Furthermore, in the preprocessing of the raw log data, only meaningless symbols (such as punctuation marks and delimiters) are removed, maximizing the semantic information of log texts. Besides, the semantics of words, logs, and log sequences are learned dynamically during training. Hence, our proposed anomaly detection framework LayerLog based on hierarchical semantics has better robustness to adapt to new log formats and is more suitable for online log anomaly detection.

5. Conclusion

Most of the existing log anomaly detection methods are based on log parsers to generate log events, log keys, or log templates. However, the Log parsers only consider the relations among words in the logs and cannot deal well with the changing logs. In this paper, we discover a three-layered structure "Word-Log-Log Sequence hierarchy" of log data and propose a novel framework for log sequence anomaly detection based on hierarchical semantics named *LayerLog*. LayerLog does not rely on any Log Parses during the preprocessing phase and can effectively extract semantic features from each layer. Besides, LayerLog has better robustness because we directly extract semantic information from raw logs instead of converting logs into log templates. We have evaluated sufficient experiments on two commonly-used public datasets and the experimental results confirm the effectiveness of LayerLog.

Table 2
Accuracy on the HDFS and BGL dataset.

Datasets	HDFS			BGL		
Methods	Precision	Recall	F1	Precision	Recall	F1
LR	0.983	0.863	0.919	1.00	0.70	0.82
SVM	1.000	0.862	0.926	0.99	0.57	0.71
PCA	1.000	0.650	0.788	0.50	0.61	0.55
IM	0.861	0.817	0.839	0.83	0.99	0.91
LogCluster	1.000	0.460	0.630	0.42	0.87	0.57
LSTM-AE	0.890	0.880	0.880	0.89	0.89	0.89
DeepLog	0.953	0.928	0.940	0.90	0.96	0.93
LogAnomaly	0.960	0.940	0.950	0.97	0.94	0.96
LayerLog(OC-SVM)	0.966	0.951	0.958	0.976	0.951	0.963
LayerLog(Classifier)	**0.996**	**0.978**	**0.988**	**0.995**	**0.974**	**0.985**

Table 3
Accuracy of online testing on BGL dataset.

	DeepLog	LogAnomaly	LayerLog
P	0.3817	0.8039	**0.9944**
R	0.9768	0.9319	**0.9187**
F1	0.5489	0.8632	**0.9550**

CRediT authorship contribution statement

Chunkai Zhang: Conceptualization, Ideas, Methodology, Writing – review & editing. **Xinyu Wang:** Investigation, Software, Validation, Writing – original draft. **Hongye Zhang:** Investigation, Software, Writing – original draft. **Jiahua Zhang:** Software, Visualization. **Hanyu Zhang:** Software, Data curation. **Chuanyi Liu:** Supervision, Writing – reviewing and editing. **Peiyi Han:** Supervision, Writing – reviewing and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by Natural Science Foundation of Guangdong Province, China (Grant NO. 2020A1515010970) and Shenzhen Research Council, China (Grant NO. JCYJ20200109113427092, GJHZ20180928155209705).

References

[1] M. Ma, S. Zhang, D. Pei, X. Huang, H. Dai, Robust and rapid adaption for concept drift in software system anomaly detection, in: 2018 IEEE 29th International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2018, pp. 13–24.

[2] R. Chalapathy, S. Chawla, Deep learning for anomaly detection: A survey, 2019, arXiv:1901.03407, [cs, stat].

[3] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, et al., Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19. International Joint Conferences on Artificial Intelligence Organization, Vol. 7, 2019, pp. 4739–4745.

[4] M. Farshchi, J.-G. Schneider, I. Weber, J. Grundy, Experience report: Anomaly detection of cloud application operations using log and cloud metric correlation analysis, in: 2015 IEEE 26th International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2015, pp. 24–34.

[5] Y. Liang, Y. Zhang, H. Xiong, R. Sahoo, Failure prediction in ibm bluegene/l event logs, in: Seventh IEEE International Conference on Data Mining (ICDM 2007), IEEE, 2007, pp. 583–588.

[6] W. Xu, L. Huang, A. Fox, D. Patterson, M.I. Jordan, Detecting large-scale system problems by mining console logs, in: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, 2009, pp. 117–132.

[7] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, J. Li, Mining invariants from console logs for system problem detection, in: 2010 USENIX Annual Technical Conference (USENIX ATC 10), 2010.

[8] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, X. Chen, Log clustering based problem identification for online service systems, in: Proceedings of the 38th International Conference on Software Engineering Companion - ICSE '16, ACM Press, Austin, Texas, 2016, pp. 102–111, http://dx.doi.org/10.1145/2889160.2889232.

[9] A. Grover, Anomaly detection for application log data (Ph.D. thesis), San Jose State University, San Jose, CA, USA, 2018, http://dx.doi.org/10.31979/etd.znsb-bw4d.

[10] M. Du, F. Li, G. Zheng, V. Srikumar, DeepLog: Anomaly detection and diagnosis from system logs through deep learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security - CCS '17, ACM Press, Dallas, Texas, USA, 2017, pp. 1285–1298, http://dx.doi.org/10.1145/3133956.3134015.

[11] S. Lu, X. Wei, Y. Li, L. Wang, Detecting anomaly in big data system logs using convolutional neural network, in: 2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech), IEEE, 2018, pp. 151–158.

[12] X. Zhang, Z. Li, J. Chen, X. He, R. Yao, J.-G. Lou, M. Chintalapati, F. Shen, D. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Robust log-based anomaly detection on unstable log data, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering - ESEC/FSE 2019, ACM Press, Tallinn, Estonia, 2019, pp. 807–817, http://dx.doi.org/10.1145/3338906.3338931.

[13] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, M.R. Lyu, Tools and benchmarks for automated log parsing, 2019, arXiv:1811.03509, [cs].

[14] W. Xu, L. Huang, A. Fox, D. Patterson, M.I. Jordan, Detecting large-scale system problems by mining console logs, in: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles - SOSP '09, ACM Press, Big Sky, Montana, USA, 2009, p. 117, http://dx.doi.org/10.1145/1629575.1629587.

[15] A. Oliner, J. Stearley, What supercomputers say: A study of five system logs, in: 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07), IEEE, Edinburgh, UK, 2007, pp. 575–584, http://dx.doi.org/10.1109/DSN.2007.103.

[16] M. Nagappan, K. Wu, M.A. Vouk, Efficiently extracting operational profiles from execution logs using suffix arrays, in: 2009 20th International Symposium on Software Reliability Engineering, IEEE, 2009, pp. 41–50.

[17] Q. Fu, J.-G. Lou, Y. Wang, J. Li, Execution anomaly detection in distributed systems through unstructured log analysis, in: 2009 Ninth IEEE International Conference on Data Mining, 2009, pp. 149–158, http://dx.doi.org/10.1109/ICDM.2009.60.

[18] Q. Fu, J.-G. Lou, Y. Wang, J. Li, Execution anomaly detection in distributed systems through unstructured log analysis, in: 2009 Ninth IEEE International Conference on Data Mining, IEEE, 2009, pp. 149–158.

[19] H. Hamooni, B. Debnath, J. Xu, H. Zhang, G. Jiang, A. Mueen, Logmine: Fast pattern recognition for log analytics, in: Proceedings of the 25th ACM International on Conference on Information and Knowledge Management, 2016, pp. 1573–1582.

[20] A.A. Makanju, A.N. Zincir-Heywood, E.E. Milios, Clustering event logs using iterative partitioning, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '09, Association for Computing Machinery, New York, NY, USA, 2009, pp. 1255–1264, http://dx.doi.org/10.1145/1557019.1557154.

[21] P. He, J. Zhu, Z. Zheng, M.R. Lyu, Drain: An online log parsing approach with fixed depth tree, in: 2017 IEEE International Conference on Web Services, ICWS, 2017, pp. 33–40, http://dx.doi.org/10.1109/ICWS.2017.13.
[22] S. Zhang, W. Meng, J. Bu, S. Yang, Y. Liu, D. Pei, J. Xu, Y. Chen, H. Dong, X. Qu, L. Song, Syslog processing for switch failure diagnosis and prediction in datacenter networks, in: 2017 IEEE/ACM 25th International Symposium on Quality of Service (IWQoS), IEEE, Vilanova i la Geltrú, Spain, 2017, pp. 1–10, http://dx.doi.org/10.1109/IWQoS.2017.7969130.
[23] M. Du, F. Li, Spell: Streaming parsing of system event logs 6.
[24] W. Meng, Y. Liu, F. Zaiter, S. Zhang, Y. Chen, Y. Zhang, Y. Zhu, E. Wang, R. Zhang, S. Tao, et al., Logparse: Making log parsing adaptive through word classification, in: 2020 29th International Conference on Computer Communications and Networks, ICCCN, IEEE, 2020, pp. 1–9.
[25] H. Dai, H. Li, C.S. Chen, W. Shang, T.-H. Chen, Logram: Efficient log parsing using n-gram dictionaries, IEEE Trans. Softw. Eng. (2020).
[26] R.B. Yadav, P.N.S. Kumar, S.V. Dhavale, A survey on log anomaly detection using deep learning, in: 2020 8th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), 2020, pp. 1215–1220.
[27] M. Chen, A.X. Zheng, J. Lloyd, M.I. Jordan, E. Brewer, Failure diagnosis using decision trees, in: International Conference on Autonomic Computing, 2004. Proceedings, IEEE, 2004, pp. 36–43.
[28] J. Fontaine, C. Kappler, A. Shahid, E.D. Poorter, Log-based intrusion detection for cloud web applications using machine learning, in: International Conference on P2P, Parallel, Grid, Cloud and Internet Computing, Springer, 2019, pp. 197–210.
[29] R. Dunia, S.J. Qin, Multi-dimensional fault diagnosis using a subspace approach, in: American Control Conference, Citeseer, 1997.
[30] P. He, J. Zhu, S. He, J. Li, M.R. Lyu, Towards automated log parsing for large-scale log data analysis, IEEE Trans. Dependable Secure Comput. 15 (6) (2018) 931–944.
[31] Q. Le, T. Mikolov, Distributed representations of sentences and documents, in: International Conference on Machine Learning, PMLR, 2014, pp. 1188–1196.
[32] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, D. Meng, Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 1777–1794.
[33] A. Joulin, E. Grave, P. Bojanowski, M. Douze, H. Jégou, T. Mikolov, Fasttext.zip: Compressing text classification models, 2016, arXiv preprint arXiv:1612.03651.
[34] H. Guo, S. Yuan, X. Wu, Logbert: Log anomaly detection via bert, in: 2021 International Joint Conference on Neural Networks, IJCNN, IEEE, 2021, pp. 1–8.
[35] X. Li, P. Chen, L. Jing, Z. He, G. Yu, Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults, in: 2020 IEEE 31st International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2020, pp. 92–103.
[36] V.-H. Le, H. Zhang, Log-based anomaly detection without log parsing, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2021, pp. 492–504.
[37] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv preprint arXiv:1810.04805.
[38] K. Toutanova, D. Klein, C.D. Manning, Y. Singer, Feature-rich part-of-speech tagging with a cyclic dependency network, in: NAACL, 2003.
[39] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, Inf. Process. Manag. 24 (1988) 513–523.
[40] T. Mikolov, K. Chen, G.S. Corrado, J. Dean, Efficient estimation of word representations in vector space, in: ICLR, 2013.
[41] E. Loper, S. Bird, NLTK: The natural language toolkit, 2002, arXiv:cs/0205028.
[42] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[43] S.M. Erfani, S. Rajasegarar, S. Karunasekera, C. Leckie, High-dimensional and large-scale anomaly detection using a linear one-class SVM with deep learning, Pattern Recognit. 58 (2016) 121–134.